AE系列篇-01.AE: 从“矩阵分解”到“数据压缩”

1. 从“矩阵分解”到“数据压缩”

1.1 矩阵分解的启示

在数学和线性代数中，矩阵分解是一个核心概念。以**奇异值分解（SVD, Singular Value Decomposition）**为例，任意矩阵 $A \in \mathbb{R}^{m \times n}$ 可以分解为:

$$A = U\Sigma V^T$$

其中: - $U \in \mathbb{R}^{m \times m}$ 和 $V \in \mathbb{R}^{n \times n}$ 是正交矩阵 - $\Sigma \in \mathbb{R}^{m \times n}$ 是对角矩阵，对角线上的元素是奇异值 $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$（$r$ 是矩阵的秩）

核心思想:在实际应用中，大矩阵中真正“有效”的信息往往只集中在少数几个大的奇异值上，而很多奇异值接近零，对应的信息是冗余的。这意味着我们可以用更少的维度来表示原始数据，而不会丢失太多信息。

例 ， 果我们只保留前 k 个最大的奇异值，可以得到一个低秩近似:

$$A \approx U_k \Sigma_k V_k^T$$

其中 U_k 和 V_k 只包含前 k 列，Σ_k 只包含前 k 个奇异值。这样就将原始 $m \times n$ 的矩阵压缩到了更小的表示空间。

具体例子:10×10矩阵的SVD压缩

让我们通过一个具体的例子来直观理解数据压缩的效果。假设我们有一个 10×10 的矩阵 A:

$$A = \begin{bmatrix} 1.2 & 0.8 & 0.6 & 0.4 & 0.3 & 0.2 & 0.1 & 0.05 & 0.02 & 0.01 \\ 0.8 & 1.5 & 1.1 & 0.7 & 0.5 & 0.3 & 0.15 & 0.08 & 0.03 & 0.01 \\ 0.6 & 1.1 & 1.8 & 1.2 & 0.8 & 0.5 & 0.25 & 0.12 & 0.05 & 0.02 \\ 0.4 & 0.7 & 1.2 & 1.6 & 1.0 & 0.6 & 0.3 & 0.15 & 0.06 & 0.02 \\ 0.3 & 0.5 & 0.8 & 1.0 & 1.4 & 0.8 & 0.4 & 0.2 & 0.08 & 0.03 \\ 0.2 & 0.3 & 0.5 & 0.6 & 0.8 & 1.2 & 0.6 & 0.3 & 0.12 & 0.04 \\ 0.1 & 0.15 & 0.25 & 0.3 & 0.4 & 0.6 & 1.0 & 0.5 & 0.2 & 0.08 \\ 0.05 & 0.08 & 0.12 & 0.15 & 0.2 & 0.3 & 0.5 & 0.8 & 0.3 & 0.12 \\ 0.02 & 0.03 & 0.05 & 0.06 & 0.08 & 0.12 & 0.2 & 0.3 & 0.6 & 0.25 \\ 0.01 & 0.01 & 0.02 & 0.02 & 0.03 & 0.04 & 0.08 & 0.12 & 0.25 & 0.5 \end{bmatrix}$$

对 A 进行SVD分解: $A = U\Sigma V^T$ ，我们得到10个奇异值（按降序排列）:

$$\sigma_1 = 5.23, \quad \sigma_2 = 2.18, \quad \sigma_3 = 1.45, \quad \sigma_4 = 0.89, \quad \sigma_5 = 0.52,$$
$$\sigma_6 = 0.28, \quad \sigma_7 = 0.15, \quad \sigma_8 = 0.08, \quad \sigma_9 = 0.04, \quad \sigma_{10} = 0.02$$

观察:前5个奇异值（5.23, 2.18, 1.45, 0.89, 0.52）占据了总能量的绝大部分，而后5个奇异值（0.28, 0.15, 0.08, 0.04, 0.02）非常小，接近零。

压缩过程:我们只保留前 $k = 5$ 个奇异值，用 $A_5 = U_5\Sigma_5 V_5^T$ 来近似原始矩阵 A。

压缩效果对比:

指标	原始矩阵 A	压缩后 A_5	压缩比
存储空间	100个元素	50个元素（U_5: 10×5, Σ_5: 5, V_5: 10×5）	**50%**
重构误差	-	$\|A - A_5\|_F = 0.35$	-
相对误差	-	$\frac{\|A-A_5\|_F}{\|A\|_F} = 2.8\%$	-
信息保留率	100%	97.2%	-

数值结果: - **压缩比**:从100个元素减少到50个元素，压缩了50% - **重构误差**:Frobenius范数误差仅为0.35 - **信息保留**:保留了97.2%的信息，仅损失2.8%

总结: 1. **大部分信息集中在少数几个大的奇异值上**:前5个奇异值就捕获了97.2%的信息 2. **小的奇异值可以安全丢弃**:后5个奇异值总共只贡献了2.8%的信息 3. **压缩是可行的**:用一

半的存储空间就能表示几乎全部的信息

这个例子清楚地展示了:**大矩阵中真正有效的信息只集中在少数几个特征值/奇异值上，很多都是零或接近零，没有意义**。这正是数据压缩和降维的核心思想，也是自编码器要解决的根本问题。

1.2 数据压缩的必要性

数据压缩是训练网络的重要阶段。核心思想是:用更少的比特来表示相同数量的信息。这在深度学习中尤为重要，原因包括:

1. **维度灾难（Curse of Dimensionality）**:具有大量属性的数据集难以训练，因为模型容易过拟合。高维空间中，数据点之间的距离变得相似，导致模型难以学习有效的模式。
2. **计算和存储成本**:高维数据需要更多的计算资源和存储空间。通过降维，可以显著减少这些成本。
3. **去除冗余信息**:真实数据中往往存在大量冗余。例 ，图像中相邻像素通常高度相关，很多信息是重复的。

1.3 自编码器的引入

正是在这样的背景下，**自编码器（Autoencoder, AE）**和**变分自编码器（Variational Autoencoder, VAE）**应运而生。它们是端到端的神经网络，用于压缩输入数据。

自编码器和变分自编码器都将数据从高维空间转换到低维空间，本质上实现了数据压缩。与传统的线性降维方法（ PCA）不同，它们通过非线性变换可以学习更强大的数据表示。

---

2. 潜在空间（Latent Space）与自编码器（AE）

2.1 什么是自编码器?

自编码器（Autoencoder, AE）是一种用于学习无标签数据高效嵌入（embeddings）的无监督学习模型。对于给定的网络配置，它可以学习数据的紧凑表示。

自编码器由两个主要部分组成:

- **编码器（Encoder）**:将高维输入数据压缩到低维潜在空间（latent space）
- **解码器（Decoder）**:将潜在表示重构回高维数据空间

解码器的作用是确保潜在空间能够捕获数据集空间中的大部分信息，通过强制其输出与编码器输入尽可能接近来实现这一点。

2.1.1 矩阵分解与数据压缩

自编码器的核心思想与矩阵分解（ SVD）一脉相承:**通过降维实现数据压缩**。让我们通过两个例子来理解这一过程。

例子1:矩阵分解（SVD）

下图展示了10×10矩阵A通过SVD分解实现压缩的过程:

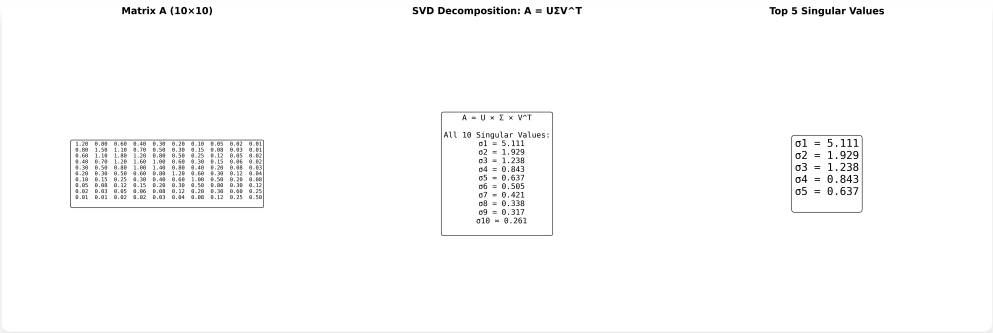


图1 左侧:原始矩阵A（10×10，高维空间） | 中间:SVD分解结果（$A = U\Sigma V^T$，显示所有10个奇异值） | 右侧:前5个最大的奇异值（低维空间）

压缩:原始矩阵 10×10 是高维空间，通过SVD分解保留前5个奇异值，压缩到低维空间。

例子2:自编码器（AE）

下图展示了MNIST图像通过自编码器实现压缩的过程:

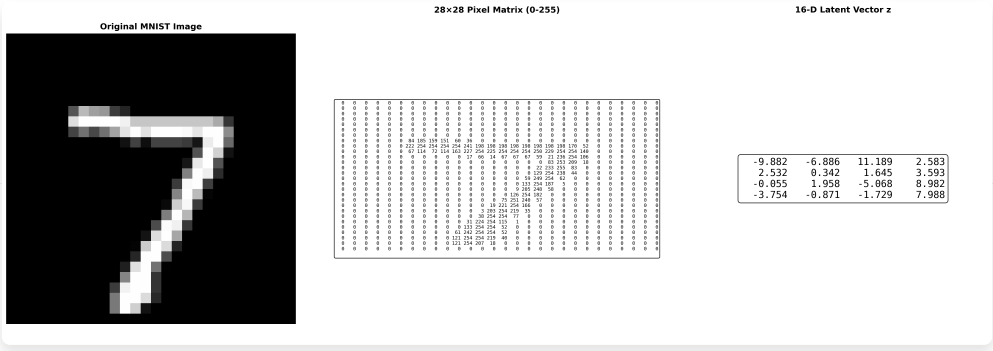


图2 左侧:原始MNIST图像（28×28像素，高维空间） | 中间:28×28数字矩阵（每个像素值0-255） | 右侧:16维潜在向量（Latent空间，低维空间）

压缩:原始图像 $28 \times 28 = 784$ 维是高维空间，通过编码器压缩到16维Latent空间（低维空间）。

2.2 架构与工作原理

自编码器由编码器和解码器两部分组成，通过编码-解码过程实现数据压缩和重构:

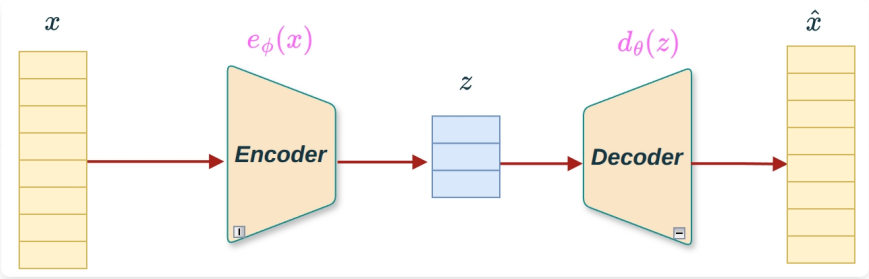


图3 自编码器架构:输入 x → 编码器 $e_\phi(x)$ → 潜在向量 z → 解码器 $d_\theta(z)$ → 重构输出 $\hat{x}$

2.3 损失函数

自编码器的目标是实现重构，因此损失函数即为重构误差，即编码器输入和解码器输出之间的均方误差:

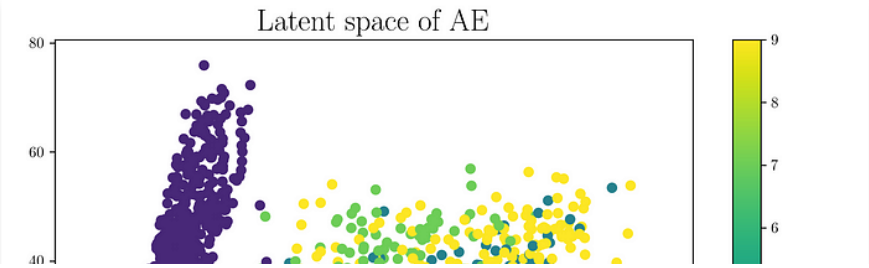
$$\mathcal{L}_{AE} = \|x - \hat{x}\|^2 = \|x - d_\theta(e_\phi(x))\|^2$$

---

3. 可视化潜在空间

3.1 潜在空间的聚类特性

下图展示了在MNIST数据集上训练自编码器后生成的潜在空间可视化示例:



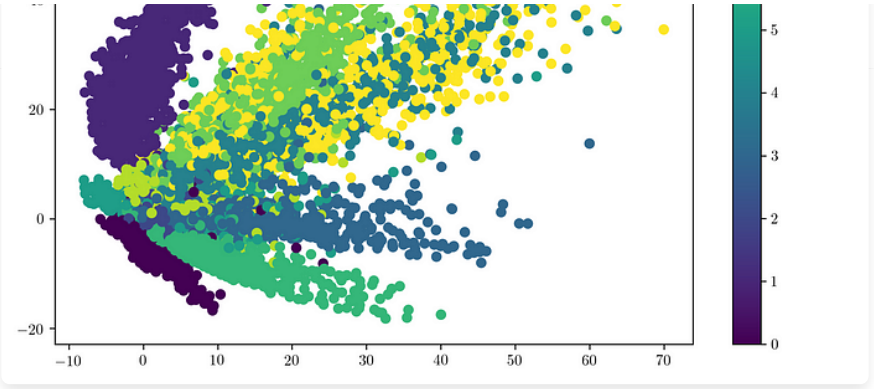


图4 潜在空间可视化:相同的数字倾向于在潜在空间中聚集在一起，形成不同的簇。同时，潜在空间中存在一些不对应任何数据点的“空洞”区域。

从图中可以观察到:

1. **聚类现象**:相同的数字倾向于在潜在空间中聚集在一起。例　，所有的“0”会形成一个簇，所有的“1”会形成另一个簇，以此类推。
2. **空洞区域**:潜在空间中存在一些不对应任何数据点的部分。　果将这些区域的值作为解码器的输入，会产生不像MNIST数据中任何数字的输出。

3.2 非正则化潜在空间的问题

这正是我们所说的**潜在空间是非正则化的（unregularized）**。这种潜在空间的特点包括:

1. **有限的生成能力**:只有少数区域/簇具有生成能力。这意味着从属于某个簇的潜在空间中采样任何点，都会生成属于该簇的数据的变体。但**整个潜在空间并不都具有生成能力**。
2. **垃圾输出区域**:不属于任何簇的区域会生成垃圾输出。一旦网络训练完成，训练数据被移除，我们无法知道从随机采样的潜在向量通过解码器生成的输出是否有效。

---

4. 代码实现

4.1 网络定义（ae.py）

自编码器的网络结构定义　下:

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class Encoder(nn.Module):
    def __init__(self, input_dim, hidden_dim, latent_dim):
        super().__init__()
        self.linear1 = nn.Linear(input_dim, hidden_dim)
        self.linear2 = nn.Linear(hidden_dim, latent_dim)

    def forward(self, x):
        h = F.relu(self.linear1(x))
        z = self.linear2(h)  # 直接输出潜在向量，不需要概率分布
        return z

class Decoder(nn.Module):
    def __init__(self, latent_dim, hidden_dim, output_dim):
        super().__init__()
        self.linear1 = nn.Linear(latent_dim, hidden_dim)
        self.linear2 = nn.Linear(hidden_dim, output_dim)

    def forward(self, z):
        h = F.relu(self.linear1(z))
        x_hat = F.sigmoid(self.linear2(h))
        return x_hat

class AE(nn.Module):
    def __init__(self, input_dim, hidden_dim, latent_dim):
        super().__init__()
        self.encoder = Encoder(input_dim, hidden_dim, latent_dim)
        self.decoder = Decoder(latent_dim, hidden_dim, input_dim)

    def forward(self, x):
        z = self.encoder(x)  # 编码:x -> z
        x_hat = self.decoder(z)  # 解码:z -> x_hat
        return x_hat

def loss_function(x_hat, x):
    # AE 只有重构损失，没有 KL 散度
    return F.mse_loss(x_hat, x, reduction='mean')
```

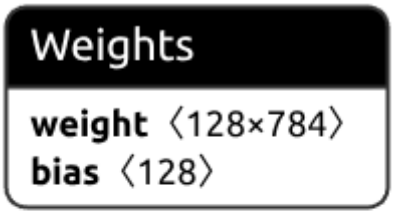




图5 自编码器网络结构

图1.5展示了自编码器网络结构，其中Encoder为两层全连接网络（使用ReLU激活函数），将输入压缩到潜在空间；Decoder为两层全连接网络（使用ReLU和Sigmoid激活函数），将潜在向量重构回原始空间。损失函数仅包含重构损失（MSE），没有正则化项。

4.2 训练与推理

4.2.1 模型训练

训练过程的核心代码　下:

```
# 模型构建
model = AE(input_dim, hidden_dim, latent_dim).to(device)
optimizer = optim.Adam(model.parameters(), lr=learning_rate)

# 训练循环
for epoch in range(epochs):
    for x, label in train_loader:
        x = x.view(-1, input_dim).to(device)
        x_hat = model(x)  # 前向传播:编码-解码
        loss = loss_function(x_hat, x)  # 计算重构损失

        optimizer.zero_grad()
```

```
        loss.backward()  # 反向传播
        optimizer.step()  # 更新参数
```

训练流程: 1. 将输入数据展平为向量（MNIST: 28×28 = 784维） 2. 通过模型进行前向传播，得到重构输出 3. 计算重构损失（MSE） 4. 反向传播更新参数

4.2.2 模型测试

训练完成后，在测试集上评估模型性能:

```
model.eval()
with torch.no_grad():
    for x, label in test_loader:
        x = x.view(-1, input_dim).to(device)
        x_hat = model(x)
        loss = loss_function(x_hat, x)
        # 记录测试损失
```

---

5. 重建能力 vs. 生成能力

5.1 推理1:重建能力（Reconstruction）

代码中的 `infer1` 部分展示了AE的**重建能力**:

```
# 模型推理1 - 从测试集采样图像，查看重构效果
if infer1:
    model.eval()
    with torch.no_grad():
        # 从测试集第一个批次中取样本
        x, labels = next(iter(test_loader))
        x_test = x[:sample_size].view(-1, input_dim).to(device)
        labels_test = labels[:sample_size]

        # 通过 AE 进行编码-解码重构
        x_reconstructed = model(x_test)

        # 转换为图像格式用于显示
        original_images = x_test.view(sample_size, 1, 28, 28).cpu()
        reconstructed_images = x_reconstructed.view(sample_size, 1,
        28, 28).cpu()
```

```
        labels_cpu = labels_test.numpy()
```

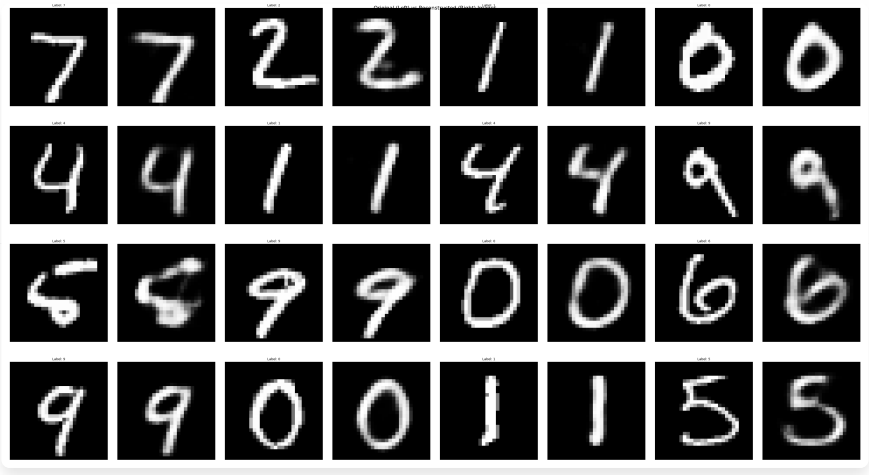


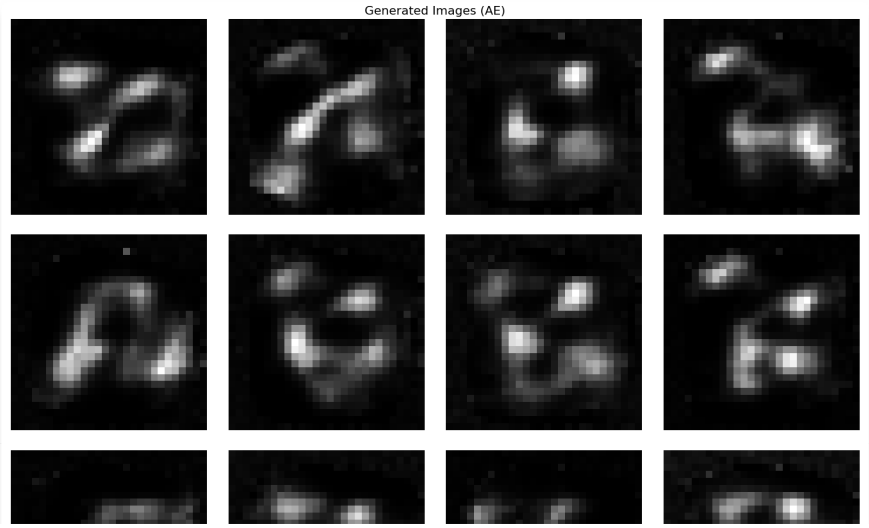
图6 AE的重建能力:左列为原始图像，右列为重构图像。

图1.6展示了AE的重建能力，两者非常相似，说明潜在空间成功捕获了输入的关键信息，证明了AE在数据压缩方面的有效性。

5.2 推理2:生成能力（Generation）

代码中的 `infer2` 部分展示了AE的**生成能力**（或更准确地说，是生成能力的局限性）：

```
# 模型推理2 - 生成新图像（从潜在空间随机采样）
if infer2:
    model.eval()
    with torch.no_grad():
        # 从标准正态分布采样潜在向量
        z = torch.randn(sample_size, latent_dim).to(device)
        x = model.decoder(z)  # 仅使用解码器生成图像
        generated_images = x.view(sample_size, 1, 28, 28).cpu()
```



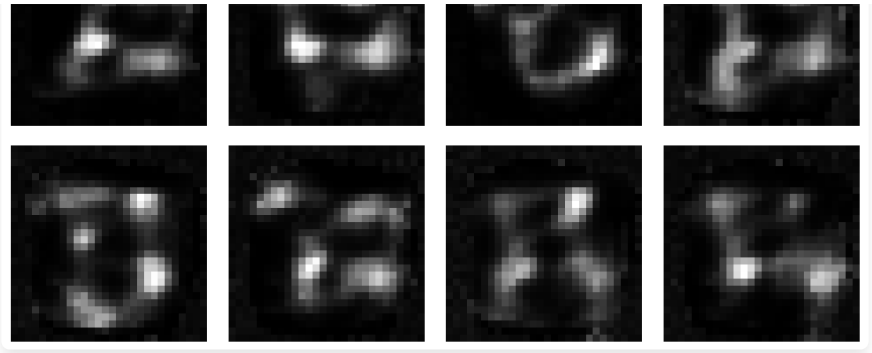


图7 AE的生成能力

图1.7展示了AE生成能力的局限性:从标准正态分布随机采样潜在向量 z，直接通过解码器生成图像。结果生成的图像通常是**无意义的**或**模糊的**，这是因为AE的潜在空间是非正则化的，存在大量“空洞”区域。

这与3中的描述是一致的，至于 何解决这个问题，我们在下一节VAE中进行介绍。